



Faculty of Engineering, Cairo University
Electronics and Electrical Communications Engineering Department
Computer Networks Course : ELC 3080
Third Year

Computer Networks Project

Virtual Networks Lab using Linux and the CORE Network Emulator

Spring 2025

Submitted to: Prof. Khaled Fouad & Dr. Rania Osama & Eng. Ahmed Atef

Name	Sec	BN	Code
رؤي عاطف محمد عبد المجيد	2	11	9220291
ساندي نجيب لبيب نجيب	2	20	9220354

Table of contents

1. Project Statement.....	4	
2. Effect of TCP Window Size.....	4	
Question 2.i: Vary iperf3 window size from 1Kbytes to 6 Kbytes in increments of 1 Kbyte, then set it to 12, 16, 24, 32 Kbytes. Plot the avg throughput and the avg number of retransmissions as function of window size. Comment on the results and explain the zigzag behavior noticed for larger window size. 4		
Question 2.ii: Click on any of the TCP a data segment whose source is node n7, dissect the segment by following different protocol layer headers from TCP->IP->Ethernet identifying how many header bytes are added by each layer, identify the TCP options used.		6
Question 2.iii: Repeat for an ACK packet sent from node n11.		7
3. TCP Short vs long paths.....	8	
Question 3.i: Compare the result of throughput with the case when connection was made to node n11.8		
4. Higher Link capacity with drops versus reliable lower capacity	9	
Question 4.i: Compare throughputs in cases a, b, c. Why b is better than c?		10
Question 4.ii: Compare throughputs in cases b, c and d. Which is better? Why?		10
Question 4.iii: Compare throughputs in cases e and f? Which is better? Why?		10
5. OSPF Link cost changes	10	
Question 5.i: Check what happens to the path between n7 and n11 ? Explain what happens		11
6. OSPF Database updates	13	
Question 6.i: Capture and explain the outputs due to the execution of step 2. Why do some destinations have more than route in the routing tables?		13
Question 6.ii: After executing step 3, determine how long it took the network to exchange link state packets and adjust routing tables. (Hint: you can calculate the required time by observing the time of first OSPF update message and the last ACK from Wireshark).		14
Question 6.iii: After execution of step 4, identify the new routing table and router database at router n2. Explain the updates in the new routing table and the new database		15

Table of Figures

Figure 1:Throughput = 1.01 Mbps and zero Retransmissions when window size is 1 KB	4
Figure 2: Throughput = 2.26 Mbps and zero retransmissions when window size is 2KB.....	4
Figure 3:Throughput = 4.73 Mbps and zero retransmissions when window size is 3KB.....	5
Figure 4:Throughput = 4.83 Mbps and zero retransmissions when window size is 4KB.....	5
Figure 5:Throughput = 5.63 Mbps and zero retransmissions when window size is 5KB.....	5
Figure 6:Throughput = 120 Kbps and 348 retransmissions when window size is 6KB	5
Figure 7:Throughput = 536 Kbps and 650 retransmissions when window size is 12KB	5
Figure 8:Throughput = 348 Kbps and 537 retransmissions when window size is 16KB	5
Figure 9:Throughput = 3.04 Mbps and 3035 retransmissions when window size is 24 KB	5
Figure 10:Throughput = 153 Kbps and 400 retransmissions when window size is 32KB	5
Figure 11: Average Throughput vs TCP window Size	6
Figure 12: Average number of Retransmissions vs TCP window size.....	6
Figure 13: TCP Packet analysis	7
Figure 14: ACK Packet details sent from node 11.....	8
Figure 15:Throughput between n7 & n8.....	8
Figure 16:Throughput between n7 & n11	8
Figure 17: Throughput = 3.96 Mbps and 1 retransmissions at Case A.....	9
Figure 18:Throughput = 2.1 Mbps and 3 retransmissions at Case B	9
Figure 19:Throughput = 523 Kbps and 127 retransmissions at Case C.....	9
Figure 20: Throughput = 236 Kbps and 123 retransmissions at Case D	9
Figure 21:Throughput = 2.61 Mbps and 45 retransmissions at Case E	10
Figure 22: Throughput = 1.79 Mbps and 80 retransmissions at Case F	10
Figure 23:Changing n5-n4 cost.....	11
Figure 24:Path between n7 & n11 before changing cost between n5-n4.....	11
Figure 25:Path between n7 & n11 after changing cost between n5-n4.....	11
Figure 26:Changing n4-n5 cost.....	12
Figure 27:Path between n7 & n11 before changing n4-n5 cost	12
Figure 28:Path between n7 & n11 after changing n4-n5 cost.....	12
Figure 29: Path from n7 to n11	13
Figure 30: Path from n11 to n7	13
Figure 31: Router n2 OSPF database.....	13
Figure 32: Router n2 OSPF route	13
Figure 33: Time stamp of the first LS update	14
Figure 34: Time stamp of the last LS Acknowledge.....	14
Figure 35: Commands disconnecting eth0 , eth1 , and eth2 from n4.....	15
Figure 36: OSPF Database for n4	15
Figure 37: OSPF Route for n4	15

1. Project Statement

This project investigates the performance of TCP in virtual networks using the CORE Network Emulator, with a focus on the impact of TCP window size, path length, link reliability, and OSPF routing dynamics.

2. Effect of TCP Window Size

The purpose of this part is to study the effect of varying the window size in TCP packets from 1 to 6 each incrementing by 1 then 12 , 16 , 24 and 32. As per studied, as window **size increase the throughput increases but up to a certain limit** , as when it increases too much this will decrease the efficiency in case of retransmission as you will re-transmit many packets even if you lost only one.

Question 2.i: Vary iperf3 window size from 1Kbytes to 6 Kbytes in increments of 1 Kbyte, then set it to 12, 16, 24, 32 Kbytes. Plot the avg throughput and the avg number of retransmissions as function of window size. Comment on the results and explain the zigzag behavior noticed for larger window size.

Table 1 : Throughput and retransmission values for different window sizes

Window Size	Throughput	Retransmissions
1 KB	1.01 Mbits/sec	0
2 KB	2.26 Mbits/sec	0
3 KB	4.73 Mbits/sec	0
4 KB	4.83 Mbits/sec	0
5 KB	5.63 Mbits/sec	0
6KB	120 Kbits/sec	348
12 KB	348 Kbits/sec	537
24 KB	3.04 Mbits /sec	3035
32 KB	153 Kbits / sec	400

```
iperf Done.
root@n7:/tmp/pycore.35470/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 1k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 43872 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00  sec  1.11 MBytes  934 Kbits/sec  0    5.45 KBytes
[ 4] 10.00-20.00  sec  1.18 MBytes  993 Kbits/sec  0    5.45 KBytes
[ 4] 20.00-30.00  sec  1.26 MBytes  1.05 Mbits/sec  0    5.45 KBytes
[ 4] 30.00-40.00  sec  1.26 MBytes  1.06 Mbits/sec  0    5.45 KBytes
-----
[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00  sec  4.82 MBytes  1.01 Mbits/sec  0
[ 4] 0.00-40.00  sec  4.81 MBytes  1.01 Mbits/sec  0
sender
receiver
iperf Done.
root@n7:/tmp/pycore.35470/n7.conf# █
```

Figure 1:Throughput = 1.01 Mbps and zero Retransmissions when window size is 1 KB

```
iperf Done.
root@n7:/tmp/pycore.35470/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 2k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 43870 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00  sec  2.14 MBytes  1.80 Mbits/sec  0    7.07 KBytes
[ 4] 10.00-20.00  sec  2.95 MBytes  2.47 Mbits/sec  0    4.24 KBytes
[ 4] 20.00-30.03  sec  2.92 MBytes  2.44 Mbits/sec  0    4.24 KBytes
[ 4] 30.03-40.02  sec  2.77 MBytes  2.33 Mbits/sec  0    4.24 KBytes
-----
[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.02  sec  10.8 MBytes  2.26 Mbits/sec  0
[ 4] 0.00-40.02  sec  10.8 MBytes  2.26 Mbits/sec  0
sender
receiver
iperf Done.
root@n7:/tmp/pycore.35470/n7.conf# █
```

Figure 2: Throughput = 2.26 Mbps and zero retransmissions when window size is 2KB

```

iperf Done.
root@n7:/tmp/pycore.35470/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 3k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 43874 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  4.23 MBytes  3.54 Mbits/sec  0    8,48 KBytes
[ 4] 10.00-20.00 sec  6.24 MBytes  5.23 Mbits/sec  0    8,48 KBytes
[ 4] 20.00-30.00 sec  6.24 MBytes  5.24 Mbits/sec  0    8,48 KBytes
[ 4] 30.00-40.00 sec  5.83 MBytes  4.89 Mbits/sec  0    8,48 KBytes

[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  22.9 MBytes  4.73 Mbits/sec  0          sender
[ 4] 0.00-40.00 sec  22.5 MBytes  4.73 Mbits/sec          receiver

```

Figure 3:Throughput = 4.73 Mbps and zero retransmissions when window size is 3KB

```

iperf Done.
root@n7:/tmp/pycore.35470/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 4k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 43876 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  4.49 MBytes  3.77 Mbits/sec  0    14.1 KBytes
[ 4] 10.00-20.00 sec  6.06 MBytes  5.08 Mbits/sec  0    14.1 KBytes
[ 4] 20.00-30.00 sec  5.92 MBytes  4.97 Mbits/sec  0    14.1 KBytes
[ 4] 30.00-40.00 sec  6.57 MBytes  5.52 Mbits/sec  0    14.1 KBytes

[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  23.1 MBytes  4.83 Mbits/sec  0          sender
[ 4] 0.00-40.00 sec  23.0 MBytes  4.83 Mbits/sec          receiver

```

Figure 4:Throughput = 4.83 Mbps and zero retransmissions when window size is 4KB

```

iperf Done.
root@n7:/tmp/pycore.35470/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 5k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 43878 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  5.86 MBytes  4.91 Mbits/sec  0    14.1 KBytes
[ 4] 10.00-20.01 sec  7.11 MBytes  5.96 Mbits/sec  0    14.1 KBytes
[ 4] 20.01-30.01 sec  6.17 MBytes  5.02 Mbits/sec  0    14.1 KBytes
[ 4] 30.01-40.08 sec  7.77 MBytes  6.64 Mbits/sec  0    14.1 KBytes

[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.08 sec  26.9 MBytes  5.63 Mbits/sec  0          sender
[ 4] 0.00-40.08 sec  26.9 MBytes  5.63 Mbits/sec          receiver

```

Figure 5:Throughput = 5.63 Mbps and zero retransmissions when window size is 5KB

```

iperf Done.
root@n7:/tmp/pycore.35470/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 6k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 43880 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  154 KBytes  126 Kbits/sec  86    4.24 KBytes
[ 4] 10.00-20.00 sec  143 KBytes  117 Kbits/sec  88    4.24 KBytes
[ 4] 20.00-30.01 sec  144 KBytes  118 Kbits/sec  87    4.24 KBytes
[ 4] 30.01-40.00 sec  143 KBytes  117 Kbits/sec  87    4.24 KBytes

[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  584 KBytes  120 Kbits/sec  348          sender
[ 4] 0.00-40.00 sec  574 KBytes  118 Kbits/sec          receiver

```

Figure 6:Throughput = 120 Kbps and 348 retransmissions when window size is 6KB

```

iperf Done.
root@n7:/tmp/pycore.35470/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 12k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 43882 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  379 KBytes  310 Kbits/sec  125    2.83 KBytes
[ 4] 10.00-20.01 sec  1.15 MBytes  965 Kbits/sec  234    8.48 KBytes
[ 4] 20.01-30.01 sec  307 KBytes  251 Kbits/sec  115    7.07 KBytes
[ 4] 30.01-40.01 sec  755 KBytes  619 Kbits/sec  176    8.48 KBytes

[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.01 sec  2.56 MBytes  536 Kbits/sec  650          sender
[ 4] 0.00-40.01 sec  2.54 MBytes  533 Kbits/sec          receiver

```

Figure 7:Throughput = 536 Kbps and 650 retransmissions when window size is 12KB

```

iperf Done.
root@n7:/tmp/pycore.35470/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 16k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 43886 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  427 KBytes  350 Kbits/sec  137    5.66 KBytes
[ 4] 10.00-20.00 sec  450 KBytes  377 Kbits/sec  140    5.66 KBytes
[ 4] 20.00-30.00 sec  424 KBytes  347 Kbits/sec  134    4.24 KBytes
[ 4] 30.00-40.00 sec  389 KBytes  319 Kbits/sec  126    4.24 KBytes

[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  1.66 MBytes  348 Kbits/sec  537          sender
[ 4] 0.00-40.00 sec  1.64 MBytes  344 Kbits/sec          receiver

```

Figure 8:Throughput = 348 Kbps and 537 retransmissions when window size is 16KB

```

iperf Done.
root@n7:/tmp/pycore.35470/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 24k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 43884 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  1.18 MBytes  992 Kbits/sec  282    4.24 KBytes
[ 4] 10.00-20.00 sec  3.83 MBytes  3.21 Mbits/sec  788    4.24 KBytes
[ 4] 20.00-30.00 sec  4.83 MBytes  4.05 Mbits/sec  1004    4.24 KBytes
[ 4] 30.00-40.00 sec  4.65 MBytes  3.90 Mbits/sec  961    2.83 KBytes

[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  14.5 MBytes  3.04 Mbits/sec  3035          sender
[ 4] 0.00-40.00 sec  14.5 MBytes  3.03 Mbits/sec          receiver

```

Figure 9:Throughput = 3.04 Mbps and 3035 retransmissions when window size is 24 KB

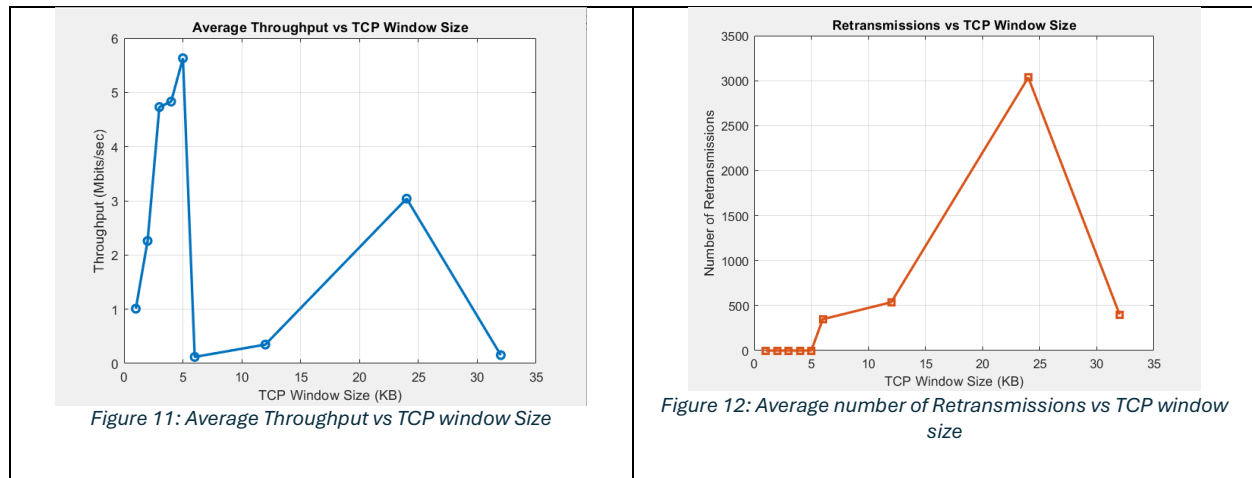
```

iperf Done.
root@n7:/tmp/pycore.35470/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 32k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 43890 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  236 KBytes  193 Kbits/sec  105    7.07 KBytes
[ 4] 10.00-20.00 sec  171 KBytes  140 Kbits/sec  90    21.2 KBytes
[ 4] 20.00-30.00 sec  171 KBytes  140 Kbits/sec  109    9.90 KBytes
[ 4] 30.00-40.00 sec  170 KBytes  139 Kbits/sec  96    12.7 KBytes

[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  748 KBytes  153 Kbits/sec  400          sender
[ 4] 0.00-40.00 sec  710 KBytes  145 Kbits/sec          receiver

```

Figure 10:Throughput = 153 Kbps and 400 retransmissions when window size is 32KB



Comments:

- The average throughput keeps increasing till it reaches the peak when window size is 5KB. When , it is 6 KB it drops, this is possibly because the receiver buffer is full, and this corresponds to a sudden increase in the number of retransmissions.
- The zigzag pattern observed starting from window size equals to 6KB is possibly that because after several retransmissions occurred the receiver increased the buffer size to accommodate to the burst data sent, so the throughput started to increase again but dropped again at 32KB because data was too large to be handled by the receiver leading to decrease in throughput again.

Question 2.ii: Click on any of the TCP a data segment whose source is node n7, dissect the segment by following different protocol layer headers from TCP->IP->Ethernet identifying how many header bytes are added by each layer, identify the TCP options used.

```

489376 506.500884 10.0.12.20 10.0.13.20 TCP 1516 [TCP Window Full] [TCP Retransmission] 43874 -> targus-getdata1 [ACK] Seq=4074710 Ack=1 Win=2920 Len=1448 TSval=79250 TSecr=79249
  Frame 489376: 1516 bytes on wire (12128 bits), 1516 bytes captured (12128 bits) on interface 0
    Arrival Time: Jun  3, 2025 07:07:56.971300000 EST
    Epoch Time: 1748923676.971300000 seconds
    [Time delta from previous captured frame: 0.000039000 seconds]
    [Time delta from previous displayed frame: 0.000039000 seconds]
    [Time since reference or first frame: 506.500884000 seconds]
    Frame Number: 489376
    Frame Length: 1516 bytes (12128 bits)
    Capture Length: 1516 bytes (12128 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ip:tcp:data]
    [Coloring Rule Name: Bad TCP]
    [Coloring Rule String: tcp.analysis.flags]

  Linux cooked capture
    Packet type: Unicast to another host (3)
    Link-layer address type: 1
    Link-layer address length: 6
    Source: 00:00:00:aa:00:1b (00:00:00:aa:00:1b)
    Protocol: IP (0x0008)

  Internet Protocol Version 4, Src: 10.0.12.20 (10.0.12.20), Dst: 10.0.13.20 (10.0.13.20)
    Version: 4
    Header length: 20 bytes
    Differentiated Services Field: 0x00 (DSCP 0x00: Default; ECN: 0x00: Not-ECT (Not ECN-Capable Transport))
    Total Length: 1500
    Identification: 0xaddc2 (56778)
    Flags: 0x02 (Don't Fragment)
    Fragment offset: 0
    Time to live: 61
    Protocol: TCP (6)
    Header checksum: 0x2822 [correct]
    Source: 10.0.12.20 (10.0.12.20)
    Destination: 10.0.13.20 (10.0.13.20)

  Transmission Control Protocol, Src Port: 43874 (43874), Dst Port: targus-getdata1 (5201), Seq: 4074710, Ack: 1, Len: 1448
    Source port: 43874 (43874)
    Destination port: targus-getdata1 (5201)
    [Stream index: 0]
    Sequence number: 4074710 (relative sequence number)
    [Next sequence number: 4076158 (relative sequence number)]
    Acknowledgement number: 1 (relative ack number)
    Header length: 32 bytes
  
```

```

Flags: 0x018 (ACK)
000. .... = Reserved: Not set
...0. .... = Nonce: Not set
.... 0. .... = Congestion Window Reduced (CWR): Not set
.... 0. .... = ECN-Echo: Not set
.... 0. .... = Urgent: Not set
.... 1. .... = Acknowledgment: Set
.... 0. .... = Push: Not set
.... 0. .... = Reset: Not set
.... 0. .... = SYN: Not set
.... 0. .... = FIN: Not set
Window size value: 2920
[Calculated window size: 2920]
[Window size scaling factor: 1]
Checksum: 0x32f6 (validation disabled)
[Good Checksum: False]
[Bad Checksum: False]
[Options: 12 bytes]
No-operation (NOP)
No-operation (NOP)
> Timestamps: TSval 79250, TSecr 79249
[Sequence/ACK analysis]
[Bytes in flight: 2896]
[TCP Analysis Flags]
P [This frame is a (suspected) retransmission]
[The RTT for this segment was: 0.002159000 seconds]
[RTO based on delta from frame: 489365]
F [The transmission window is now completely full]
Data (1448 bytes)

```

Figure 13: TCP Packet analysis

Comments:

- Frame length equals 1516 bytes
- IP Header length equals 20 bytes
- TCP Options equals 12 bytes (2 bytes for NOP and 10 for time stamps)
- TCP Header length equals 32 bytes (12 bytes options + 20 bytes without options)
- Ethernet header length equals = Frame length – IP header length – TCP header length = 1516 – 20 – 32 = 1464 bytes
- TCP Options used: 2 NOP and 10 Time stamps

Question 2.iii: Repeat for an ACK packet sent from node n11.

```

489379 506.502168 10.0.13.20 10.0.12.20 TCP 68 [TCP Dup ACK 489377#1] targus-getdata1 > 43874 [ACK] Seq=1 Ack=4074710 Win=2896 Len=0 TSval=79251 TSecr=79250
Frame 489379: 68 bytes on wire (544 bits), 68 bytes captured (544 bits)
Arrival Time: Jun 3, 2025 07:07:56.973204000 EEST
Epoch Time: 1748923676.973204000 seconds
[Time delta from previous captured frame: 0.000194000 seconds]
[Time delta from previous displayed frame: 0.000194000 seconds]
[Time since reference or first frame: 506.502168000 seconds]
Frame Number: 489379
Frame Length: 68 bytes (544 bits)
Capture Length: 68 bytes (544 bits)
[Frame is marked: False]
[Frame is ignored: False]
[Protocols in frame: sll:ip:tcp]
[Coloring Rule Name: Bad TCP]
[Coloring Rule String: tcp.analysis.flags]
Linux cooked capture
Packet type: Sent by us (4)
Link-layer address type: 1
Link-layer address length: 6
Source: 08:00:00_aa:00:1a (08:00:00:aa:00:1a)
Protocol: IP (0x0800)
Internet Protocol Version 4, Src: 10.0.13.20 (10.0.13.20), Dst: 10.0.12.20 (10.0.12.20)
Version: 4
Header length: 20 bytes
Differential Services Field: 0x00 (DSCP 0x00: Default; ECN: 0x00: Not-ECT (Not ECN-Capable Transport))
Total length: 52
Identification: 0x0051 (01)
Flags: 0x02 (Don't Fragment)
Fragment offset: 0
Time to live: 64
Protocol: TCP (6)
Header checksum: 0x0d4c [correct]
Source: 10.0.13.20 (10.0.13.20)
Destination: 10.0.12.20 (10.0.12.20)
Transmission Control Protocol, Src Port: targus-getdata1 (5201), Dst Port: 43874 (43874), Seq: 1, Ack: 4074710, Len: 0
Source port: targus-getdata1 (5201)
Destination port: 43874 (43874)
[Stream index: 8]
Sequence number: 1 (relative sequence number)
Acknowledgement number: 4074710 (relative ack number)
Header length: 32 bytes

```

```

Flags: 0x010 (ACK)
000. .... = Reserved: Not set
...0. .... = Nonce: Not set
...0. .... = Congestion Window Reduced (CWR): Not set
...0. .... = ECN-Echo: Not set
...0. .... = Urgent: Not set
...1. .... = Acknowledgement: Set
...0. .... = Push: Not set
...0. .... = Reset: Not set
...0. .... = Syn: Not set
...0. .... = Fin: Not set
Window size value: 2896
[Calculated window size: 2896]
[Window size scaling factor: 1]
Checksum: 0x2d4e [validation disabled]
[Good Checksum: False]
[Bad Checksum: False]
Options: (12 bytes)
No-Operation (NOP)
No-Operation (NOP)
Timestamps: TSval 79251, TSecr 79250
Kind: Timestamp (0)
Length: 10
Timestamp value: 79251
Timestamp echo reply: 79250
[SEQ/ACK analysis]
[TCP Analysis Flags]
[This is a TCP duplicate ack]
[Duplicate ACK #: 1]
[Duplicate to the ACK in frame: 489377]
[Expert Info (Note/Sequence): Duplicate ACK (#1)]

```

Figure 14: ACK Packet details sent from node 11

Comments:

- Frame length equals 68 bytes
- IP Header length equals 20 bytes
- TCP options equal 12 bytes (2 bytes for NOP and 10 for time stamps)
- TCP Header length equals 32 bytes (12 bytes options + 20 bytes without options)
- Ethernet header length equals = Frame length – IP header length – TCP header length = $68 - 20 - 32 = 16$ bytes
- TCP Options used: 2 NOP and 10 Time stamps

3. TCP Short vs long paths

The aim of this task is to illustrate how the length of a network path affects TCP performance specifically throughput, even when the link capacity is the same. The task is to compare the throughput of both cases when client n7 was connected to n8 and when connected to n11.

Question 3.i: Compare the result of throughput with the case when connection was made to node n11.

```

root@n7:/tmp/pycore,43563/n7.conf# iperf3 -c 10.0.11.20 -t 40 -i 10 -w 4k
Connecting to host 10.0.11.20, port 5201
[ 4] local 10.0.12.20 port 34847 connected to 10.0.11.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  5.51 MBytes  4.62 Mbits/sec  0    14.1 KBytes
[ 4] 10.00-20.00 sec  4.95 MBytes  4.15 Mbits/sec  0    14.1 KBytes
[ 4] 20.00-30.02 sec  3.79 MBytes  3.17 Mbits/sec  0    14.1 KBytes
[ 4] 30.02-40.00 sec  3.74 MBytes  3.14 Mbits/sec  0    14.1 KBytes
-----
[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  18.0 MBytes  3.77 Mbits/sec  0
[ 4] 0.00-40.00 sec  18.0 MBytes  3.77 Mbits/sec
iperf Done.
root@n7:/tmp/pycore,43563/n7.conf#

```

Figure 15:Throughput between n7 & n8

```

iperf Done.
root@n7:/tmp/pycore,35470/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 4k
Connecting to host 10.0.13.20, port 5201
[ 4] local 10.0.12.20 port 43876 connected to 10.0.13.20 port 5201
[ ID] Interval      Transfer    Bandwidth  Retr  Cwnd
[ 4] 0.00-10.00 sec  4.49 MBytes  3.77 Mbits/sec  0    14.1 KBytes
[ 4] 10.00-20.00 sec  6.06 MBytes  5.08 Mbits/sec  0    14.1 KBytes
[ 4] 20.00-30.00 sec  5.92 MBytes  4.97 Mbits/sec  0    14.1 KBytes
[ 4] 30.00-40.00 sec  6.57 MBytes  5.52 Mbits/sec  0    14.1 KBytes
-----
[ ID] Interval      Transfer    Bandwidth  Retr
[ 4] 0.00-40.00 sec  23.1 MBytes  4.83 Mbits/sec  0
[ 4] 0.00-40.00 sec  23.0 MBytes  4.83 Mbits/sec
iperf Done.
root@n7:/tmp/pycore,35470/n7.conf#

```

Figure 16:Throughput between n7 & n11

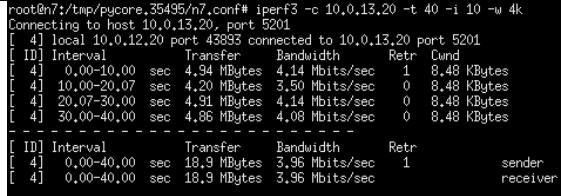
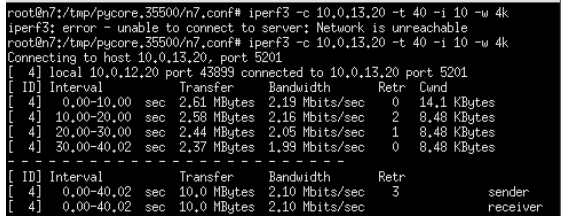
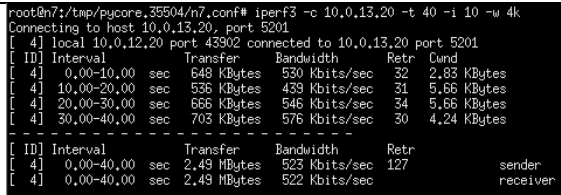
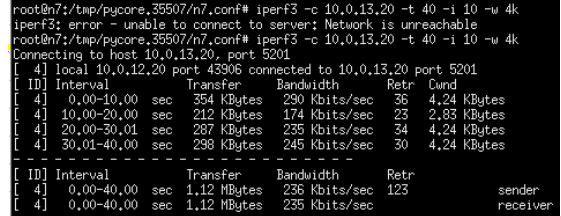
Comment:

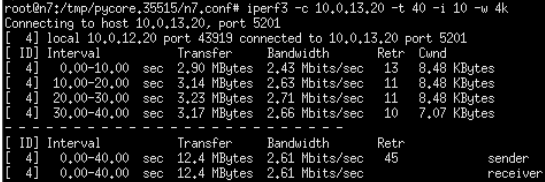
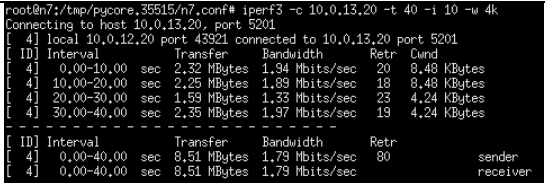
The throughput between **n7** and **n8** is about **3.77 Mbps**, while between **n7** and **n11** it is around **4.83 Mbps**. This shows that the connection to **n11** performs better. The main reason for the lower throughput to **n8** is that the path between **n7** and **n8** is longer. It likely passes through more devices and takes more time, which increases delay. Even though both paths may have the same capacity, the longer path to **n8** makes data transfer less efficient, leading to lower throughput.

4. Higher Link capacity with drops versus reliable lower capacity

This part is based on the path between n7 and n11. It is required to vary the capacity and loss parameter between specific nodes and check the effect on the throughput between nodes 11 and 7.

Table 2: Bandwidth and number of retransmissions for each different case of loss and capacity

Case	Throughput and Retransmission	Figure
A: Link between n4 and n5 having capacity of 10Mbps and zero loss	- Throughput = 3.96 Mbps - Retransmissions = 1	 <pre>root@n7:/tmp/pycore.35495/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 4k Connecting to host 10.0.13.20, port 5201 [4] local 10.0.12.20 port 43893 connected to 10.0.13.20 port 5201 [ID] Interval Transfer Bandwidth Retr Cwnd [4] 0.00-10.00 sec 4.94 MBytes 4.14 Mbits/sec 1 8.48 KBytes [4] 10.00-20.07 sec 4.20 MBytes 3.50 Mbits/sec 0 8.48 KBytes [4] 20.07-30.00 sec 4.31 MBytes 4.14 Mbits/sec 0 8.48 KBytes [4] 30.00-40.00 sec 4.86 MBytes 4.08 Mbits/sec 0 8.48 KBytes ----- [ID] Interval Transfer Bandwidth Retr [4] 0.00-40.00 sec 18.9 MBytes 3.96 Mbits/sec 1 sender [4] 0.00-40.00 sec 18.9 MBytes 3.96 Mbits/sec receiver</pre> Figure 17: Throughput = 3.96 Mbps and 1 retransmissions at Case A
B: Link between n4 and n5 having capacity of 3Mbps and zero loss	- Throughput = 2.10 Mbps - Retransmissions = 3	 <pre>root@n7:/tmp/pycore.35500/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 4k iperf3: error - unable to connect to server: Network is unreachable root@n7:/tmp/pycore.35500/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 4k Connecting to host 10.0.13.20, port 5201 [4] local 10.0.12.20 port 43899 connected to 10.0.13.20 port 5201 [ID] Interval Transfer Bandwidth Retr Cwnd [4] 0.00-10.00 sec 2.61 MBytes 2.19 Mbits/sec 0 14.1 KBytes [4] 10.00-20.00 sec 2.59 MBytes 2.16 Mbits/sec 2 8.48 KBytes [4] 20.00-30.00 sec 2.44 MBytes 2.05 Mbits/sec 1 8.48 KBytes [4] 30.00-40.02 sec 2.37 MBytes 1.99 Mbits/sec 0 8.48 KBytes ----- [ID] Interval Transfer Bandwidth Retr [4] 0.00-40.02 sec 10.0 MBytes 2.10 Mbits/sec 3 sender [4] 0.00-40.02 sec 10.0 MBytes 2.10 Mbits/sec receiver</pre> Figure 18: Throughput = 2.1 Mbps and 3 retransmissions at Case B
C: Link between n4 and n5 having capacity 10 Mbps and 5% loss	- Throughput = 523 Kbps - Retransmissions = 127	 <pre>root@n7:/tmp/pycore.35504/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 4k Connecting to host 10.0.13.20, port 5201 [4] local 10.0.12.20 port 43902 connected to 10.0.13.20 port 5201 [ID] Interval Transfer Bandwidth Retr Cwnd [4] 0.00-10.00 sec 648 KBytes 530 Kbits/sec 32 2.83 KBytes [4] 10.00-20.00 sec 536 KBytes 439 Kbits/sec 31 5.66 KBytes [4] 20.00-30.00 sec 666 KBytes 546 Kbits/sec 34 5.66 KBytes [4] 30.00-40.00 sec 703 KBytes 576 Kbits/sec 30 4.24 KBytes ----- [ID] Interval Transfer Bandwidth Retr [4] 0.00-40.00 sec 2.49 MBytes 523 Kbits/sec 127 sender [4] 0.00-40.00 sec 2.49 MBytes 522 Kbits/sec receiver</pre> Figure 19: Throughput = 523 Kbps and 127 retransmissions at Case C
D: Link between n4 and n5 having capacity 100 Mbps and 10% loss	- Throughput= 236 Kbps - Retransmissions=123	 <pre>root@n7:/tmp/pycore.35507/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 4k iperf3: error - unable to connect to server: Network is unreachable root@n7:/tmp/pycore.35507/n7.conf# iperf3 -c 10.0.13.20 -t 40 -i 10 -w 4k Connecting to host 10.0.13.20, port 5201 [4] local 10.0.12.20 port 43906 connected to 10.0.13.20 port 5201 [ID] Interval Transfer Bandwidth Retr Cwnd [4] 0.00-10.00 sec 354 KBytes 290 Kbits/sec 36 4.24 KBytes [4] 10.00-20.00 sec 212 KBytes 174 Kbits/sec 23 2.83 KBytes [4] 20.00-30.01 sec 287 KBytes 235 Kbits/sec 34 4.24 KBytes [4] 30.01-40.00 sec 238 KBytes 245 Kbits/sec 30 4.24 KBytes ----- [ID] Interval Transfer Bandwidth Retr [4] 0.00-40.00 sec 1.12 MBytes 236 Kbits/sec 123 sender [4] 0.00-40.00 sec 1.12 MBytes 235 Kbits/sec receiver</pre> Figure 20: Throughput = 236 Kbps and 123 retransmissions at Case D

E: Link between n4 and n5 having capacity of 10 Mbps and 1% loss in n4 to n5 and 0% other direction	- Throughput = 2.61 Mbps - Retransmissions = 45	 Figure 21: Throughput = 2.61 Mbps and 45 retransmissions at Case E
F: Link between n4 and n5 having capacity of 10 Mbps with 0% loss from n4 to n5 and 1% loss in other direction	- Throughput = 1.79 Mbps - Retransmissions = 80	 Figure 22: Throughput = 1.79 Mbps and 80 retransmissions at Case F

Question 4.i: Compare throughputs in cases a, b, c. Why b is better than c?

As observed from Table 2, case A has the highest throughput, which is 3.96 Mbps followed by case B and then case C. Case A represents the ideal scenario, which is the highest capacity and no losses, so it is expected to have the highest throughput. Although case C has a higher capacity than case B, case B has a higher throughput. **This is due to the loss factor in case C** is 5% unlike case B with zero losses. This factor causes the number of retransmissions to increase from 3 in case B to 127 in case C, thus the throughput decreases in case C.

Question 4.ii: Compare throughputs in cases b, c and d. Which is better? Why?

Case b has the highest throughput, which is 2.10 Mbps as compared to case c and d. Case d has the lowest throughput although it has the highest capacity as compared to b and c. This is because it has high losses that equals 10%. We conclude that it is not always the best case to have the highest possible capacity if the connection is not reliable and it has a lot of losses.

Question 4.iii: Compare throughputs in cases e and f? Which is better? Why?

Although both cases have the same capacity and same loss value but in different directions, **case e has higher throughput** and lower retransmissions. This is possibly because n7 is the client that **sends data** to the server, data takes the path from n5 to n4 while when the server replies it sends ACKs that take the path from n4 to n5. ACKs are usually smaller than data, so their loss does not affect the throughput as much as the loss and retransmissions of actual data.

5. OSPF Link cost changes

The aim of this task is to analyze how OSPF deals with link cost changes by recalculating paths and rerouting traffic accordingly.

Question 5.i: Check what happens to the path between n7 and n11 ? Explain what happens

```
root@n5:/tmp/pycore.43563/n5.conf# vtysh
Hello, this is Quagga (version 0.99.20.1).
copyright 1996-2005 Kunihiko Ishiguro, et al.

n5# show ip ospf interface eth1
eth1 is up
  I/F index 35, MTU 1500 bytes, BW 0 Kbit <UP, BROADCAST, RUNNING, MULTICAST>
  Internet Address 10.0.4.2/24, Area 0.0.0.0
  MTU mismatch detection:enabled
  Router ID 10.0.5.1, Network Type BROADCAST, Cost: 10
  Transmit Delay is 1 sec, State DR, Priority 1
  Designated Router (ID) 10.0.5.1, Interface Address 10.0.4.2
  Backup Designated Router (ID) 10.0.3.1, Interface Address 10.0.4.1
  Saved Network-LSN sequence number 0x80000002
  Multicast group memberships: OSPFv1Routers OSPFDesignatedRouters
  Timer intervals configured, Hello 10s, Dead 40s, Wait 40s, Retransmit 5
  Hello due in 6.980s
  Neighbor Count is 1, Adjacent neighbor count is 1
n5# configure terminal
n5(config)# interface eth1
n5(config-if)# ospf cost 40
n5(config-if)#
```

Figure 23: Changing n5-n4 cost

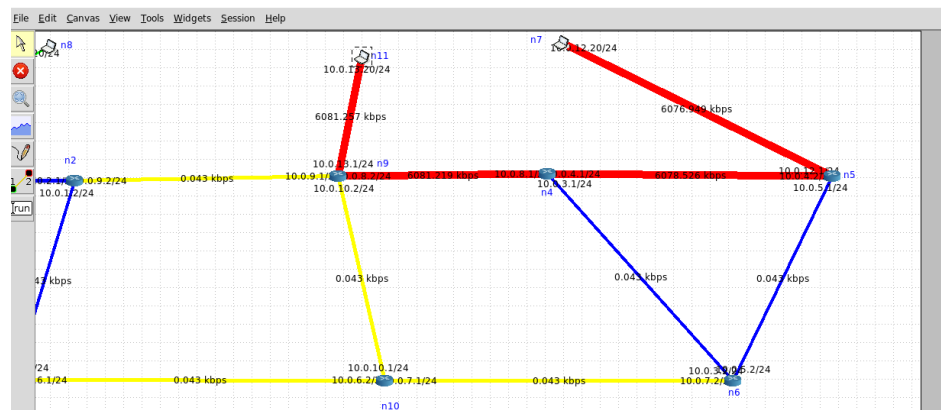


Figure 24: Path between n7 & n11 before changing cost between n5-n4

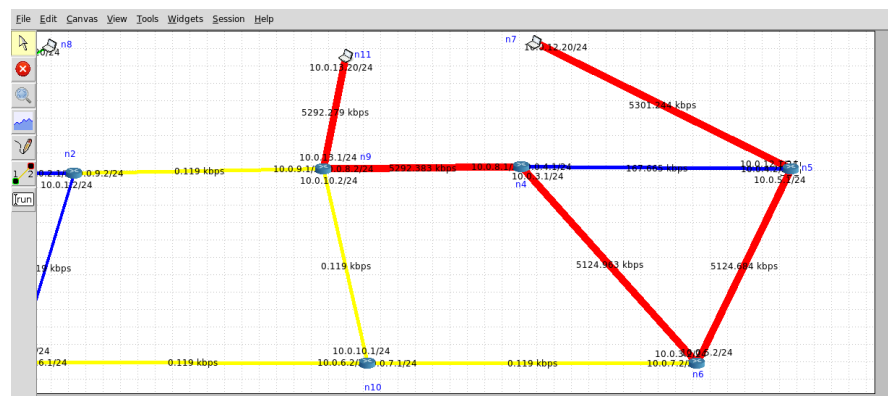


Figure 25: Path between n7 & n11 after changing cost between n5-n4

Comment:

- When the cost between n5 and n4 was changed to 40, the router running Dijkstra's algorithm automatically selected a new path from n7 to n11 with a lower total cost. Since Dijkstra's algorithm always chooses the **shortest path**, it avoided the more expensive route through n5.

Question 5.ii: Set the cost of eth1 at node n5 back to 10. Establish two iperf3 connections: one from n7 to n11 and the second from n11 to n7 both for duration of 500 seconds. Now go to node n4 and set interface cost for interface connecting n4 with n5 to 40. What happens in the paths of the two connections? Explain what happens. What do you conclude?

```

vcmd
root@n4:/tmp/pycore.43563/n4.conf# vtysh
Hello, this is Quagga (version 0.99.20.1).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

n4# show ip ospf interface eth1
eth1 is up
  ifindex 37, MTU 1500 bytes, BW 0 Kbit <UP,BROADCAST,RUNNING,MULTICAST>
  Internet Address 10.0.4.1/24, Area 0.0.0.0
  MTU mismatch detection:enabled
  Router ID 10.0.3.1, Network Type BROADCAST, Cost: 10
  Transmit Delay is 1 sec, State Backup, Priority 1
  Designated Router (ID) 10.0.5.1, Interface Address 10.0.4.2
  Backup Designated Router (ID) 10.0.3.1, Interface Address 10.0.4.1
  Multicast group memberships: OSPFAllRouters OSPFDesignatedRouters
  Timer intervals configured, Hello 10s, Dead 40s, Wait 40s, Retransmit 5
  Hello due in 5.220s
  Neighbor Count is 1, Adjacent neighbor count is 1
n4# configure terminal
n4(config)# interface eth1
n4(config-if)# ospf cost 40
n4(config-if)#

```

Figure 26: Changing n4-n5 cost

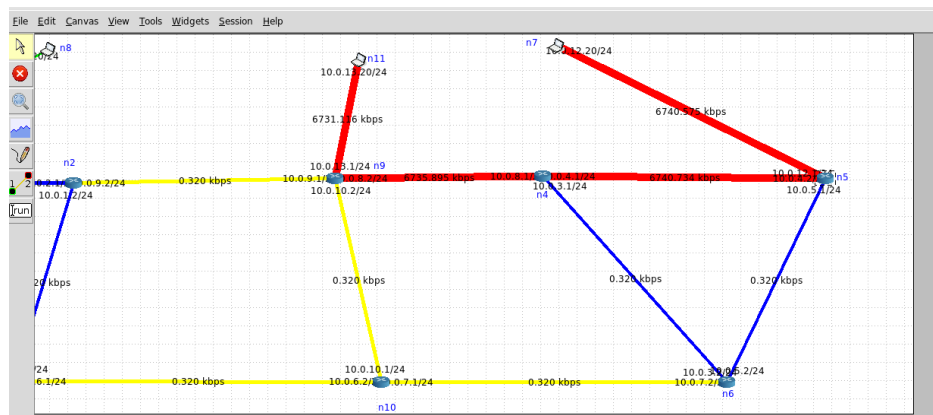


Figure 27: Path between n7 & n11 before changing n4-n5 cost

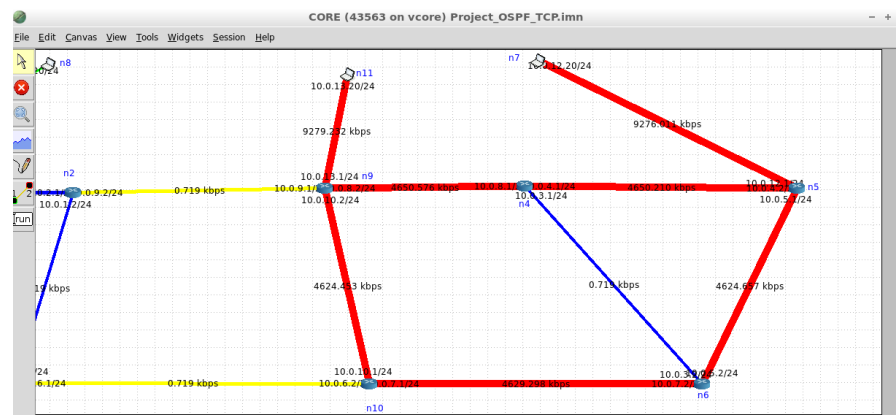


Figure 28: Path between n7 & n11 after changing n4-n5 cost

Comment

- When changing the cost between n4 and n5 to 40, the router running Dijkstra's algorithm automatically selected a new path from n7 to n11 and from n11 to n7. with a lower total cost. Since Dijkstra's algorithm always chooses the shortest path, it avoided the more expensive route through n5.

- It is observed that the path from n7 to n11 is different from the path from n11 to n7 because the router uses Dijkstra's algorithm to find the best path each time.

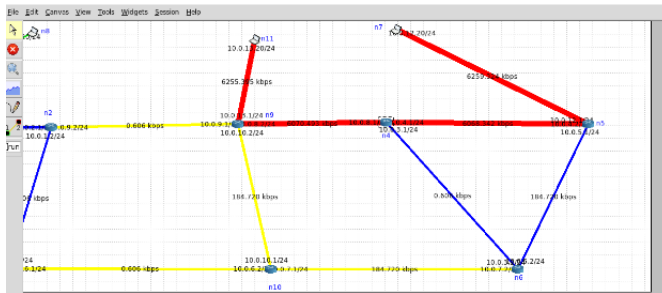


Figure 29: Path from n7 to n11

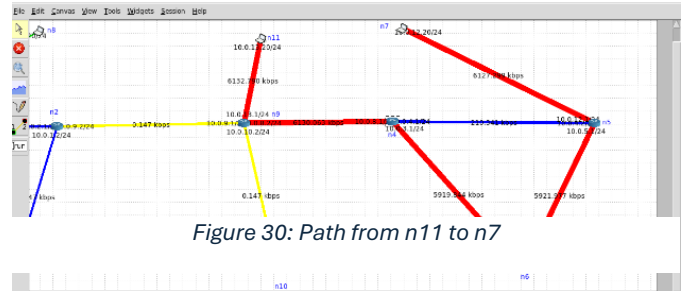


Figure 30: Path from n11 to n7

- In conclusion:** The router is responsible each time for choosing the shortest path that has a lower overall cost using Dijkstra algorithm to achieve the best performance

6. OSPF Database updates

The purpose of this task is to analyze how OSPF reacts to network changes such as changing the cost of some links and the failure of some interfaces.

Question 6.i: Capture and explain the outputs due to the execution of step 2. Why do some destinations have more than route in the routing tables?

This part includes the output of execution of two commands that are “show ip ospf database” and “show ip ospf route”. Those commands aim to show the actual OSPF routing table based on the current link state database. Also, they show link state database which is the complete map of the network that each router maintains.

```
root@n2:/tmp/pycore.47876/n2.conf# vtysh
Hello, this is Quagga (version 0.99.20.1).
Copyright 1996-2005 Kunihiro Ishiguro, et al.

n2# show ip ospf database

    OSPF Router with ID (10.0.1.2)

      Router Link States (Area 0.0.0.0)

Link ID      ADV Router   Age Seq#       DkSum Link count
10.0.0.1     10.0.0.1     307 0x8000000a 0xf2bd 3
10.0.0.2     10.0.0.2     300 0x8000000a 0x9a08 3
10.0.1.2     10.0.1.2     297 0x8000000a 0x8d0a 3
10.0.3.1     10.0.3.1     304 0x8000000a 0xbfcf 3
10.0.3.2     10.0.3.2     306 0x8000000b 0xabdf 3
10.0.5.1     10.0.5.1     304 0x80000008 0x9405 3
10.0.6.2     10.0.6.2     299 0x80000009 0xccab 3
10.0.8.2     10.0.8.2     298 0x8000000b 0x2414 4

      Net Link States (Area 0.0.0.0)

Link ID      ADV Router   Age Seq#       DkSum
10.0.0.2     10.0.0.2     309 0x80000001 0x5fcb
10.0.1.2     10.0.1.2     297 0x80000001 0x64c2
10.0.2.1     10.0.1.2     307 0x80000001 0x51d6
10.0.3.2     10.0.3.2     305 0x80000001 0x6bb3
10.0.4.2     10.0.5.1     305 0x80000001 0x64b7
...skipping one line
10.0.6.2     10.0.6.2     310 0x80000001 0x4bcc
10.0.7.1     10.0.6.2     310 0x80000001 0x59bb
10.0.8.2     10.0.8.2     308 0x80000001 0x52bd
10.0.9.1     10.0.8.2     308 0x80000001 0x2de3
10.0.10.2    10.0.8.2     298 0x80000001 0x61a8
```

Figure 31: Router n2 OSPF database

```
n2# show ip ospf route

===== OSPF network routing table =====
N   10.0.0.0/24      [20] area: 0.0.0.0
    via 10.0.1.1, eth0
N   10.0.1.0/24      [10] area: 0.0.0.0
    directly attached to eth0
N   10.0.2.0/24      [10] area: 0.0.0.0
    directly attached to eth1
N   10.0.3.0/24      [30] area: 0.0.0.0
    via 10.0.9.1, eth2
N   10.0.4.0/24      [30] area: 0.0.0.0
    via 10.0.9.1, eth2
N   10.0.5.0/24      [40] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.9.1, eth2
N   10.0.6.0/24      [20] area: 0.0.0.0
    via 10.0.1.1, eth0
N   10.0.7.0/24      [30] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.9.1, eth2
N   10.0.8.0/24      [20] area: 0.0.0.0
    via 10.0.9.1, eth2
N   10.0.9.0/24      [10] area: 0.0.0.0
```

Figure 32: Router n2 OSPF route

Figure 1 Explanation:

- When analyzing the first line it illustrates that router 10.0.0.1 received the last LSA 307 seconds ago and from the link count column, it has 3 links. LSA is a type of message used by OSPF routing protocol to exchange routing information. The same applies for the rest of the lines.
- It also has a checksum field to verify the integrity of each LSA.
- The graph is categorized into “Router Link states” and “Net Link states”. Router link states are generated by each router, and they describe each router and the links it advertises. Net link states are generated by designated routers and describe the routers attached to a shared network

Figure 2 Explanation

- The left column represents the destination address. The right column is how to reach the destination address.
- It is observed that for some destination addresses, they can be reached in two ways. This is because **both links have the same cost**. OSPF support Equal-cost Multi path to improve reliability of the network in case any of the links are down.
- The left column is also displaying the cost of each link between two parentheses as [20] , [10] ...etc.

Question 6.ii: After executing step 3, determine how long it took the network to exchange link state packets and adjust routing tables. (Hint: you can calculate the required time by observing the time of first OSPF update message and the last ACK from Wireshark).

No.	Time	Source	Destination	Protocol	Length	Info
747	90.026611	10.0.11.1	224.0.0.5	OSPF	80	Hello Packet
748	90.030614	10.0.0.1	224.0.0.5	OSPF	84	Hello Packet
749	90.030614	10.0.2.2	224.0.0.5	OSPF	84	Hello Packet
750	90.030614	10.0.11.1	224.0.0.5	OSPF	80	Hello Packet
751	92.447973	10.0.3.1	224.0.0.5	OSPF	124	LS Update
752	92.447973	10.0.3.1	224.0.0.5	OSPF	124	LS Update
753	92.447973	10.0.4.1	224.0.0.5	OSPF	124	LS Update
754	92.447973	10.0.4.1	224.0.0.5	OSPF	124	LS Update
755	92.447973	10.0.8.1	224.0.0.5	OSPF	124	LS Update
756	92.447973	10.0.8.1	224.0.0.5	OSPF	124	LS Update
757	92.451975	10.0.8.1	224.0.0.5	OSPF	124	LS Update

Figure 33: Time stamp of the first LS update

821	92.972267	10.0.5.2	224.0.0.5	OSPF	80	LS Acknowledge
822	92.972267	10.0.3.2	224.0.0.5	OSPF	80	LS Acknowledge
823	92.976270	10.0.2.2	224.0.0.5	OSPF	80	LS Acknowledge
824	92.976270	10.0.2.2	224.0.0.5	OSPF	80	LS Acknowledge
825	92.976270	10.0.0.1	224.0.0.5	OSPF	80	LS Acknowledge
826	92.976270	10.0.0.1	224.0.0.5	OSPF	80	LS Acknowledge
827	92.980272	10.0.2.2	224.0.0.5	OSPF	80	LS Acknowledge
828	92.980272	10.0.0.1	224.0.0.5	OSPF	80	LS Acknowledge
829	100.016227	10.0.8.1	224.0.0.5	OSPF	84	Hello Packet
830	100.016227	10.0.8.1	224.0.0.5	OSPF	84	Hello Packet
831	100.016227	10.0.4.1	224.0.0.5	OSPF	84	Hello Packet
832	100.016227	10.0.4.1	224.0.0.5	OSPF	84	Hello Packet
833	100.016227	10.0.3.1	224.0.0.5	OSPF	84	Hello Packet
834	100.016227	10.0.3.1	224.0.0.5	OSPF	84	Hello Packet

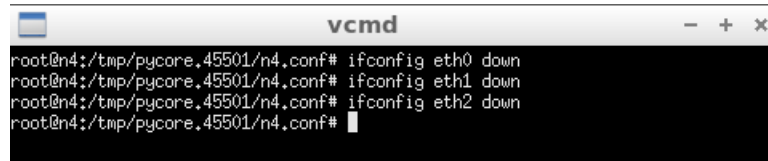
Figure 34: Time stamp of the last LS Acknowledge

Comments:

As seen from the attached pictures, the time needed to exchange link state packets can be calculated as follows:

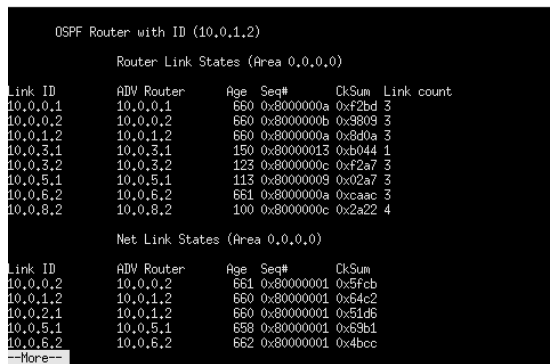
$$\text{Time of last ACK} - \text{Time of first LS update} = 92.980272 - 92.447973 = 0.534747 \text{ seconds}$$

Question 6.iii: After execution of step 4, identify the new routing table and router database at router n2. Explain the updates in the new routing table and the new database



```
root@n4:/tmp/pycore.45501/n4.conf# ifconfig eth0 down
root@n4:/tmp/pycore.45501/n4.conf# ifconfig eth1 down
root@n4:/tmp/pycore.45501/n4.conf# ifconfig eth2 down
root@n4:/tmp/pycore.45501/n4.conf#
```

Figure 35: Commands disconnecting eth0 , eth1 , and eth2 from n4



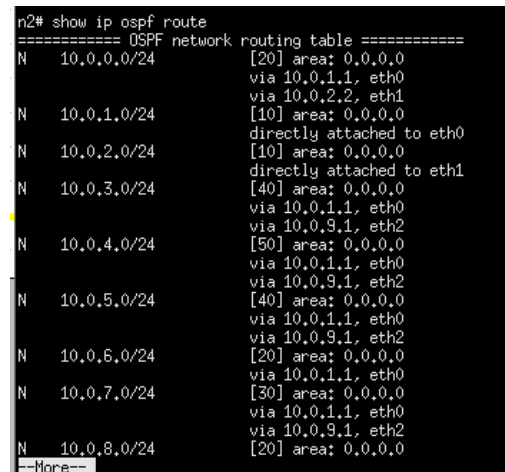
```
OSPF Router with ID (10.0.1.2)
Router Link States (Area 0.0.0.0)

Link ID      ADV Router   Age  Seq#       CkSum  Link count
10.0.0.1     10.0.0.1     660  0x8000000a 0xf2bd 3
10.0.0.2     10.0.0.2     660  0x8000000b 0x9809 3
10.0.1.2     10.0.1.2     660  0x8000000a 0x8d0a 3
10.0.3.1     10.0.3.1     150  0x80000013 0xb044 1
10.0.3.2     10.0.3.2     123  0x8000000c 0xf2a7 3
10.0.5.1     10.0.5.1     113  0x80000009 0x02a7 3
10.0.6.2     10.0.6.2     661  0x8000000a 0xcaac 3
10.0.8.2     10.0.8.2     100  0x8000000c 0x2a22 4

Net Link States (Area 0.0.0.0)

Link ID      ADV Router   Age  Seq#       CkSum
10.0.0.2     10.0.0.2     661  0x80000001 0x5fcb
10.0.1.2     10.0.1.2     660  0x80000001 0x84e2
10.0.2.1     10.0.1.2     660  0x80000001 0x51d6
10.0.5.1     10.0.5.1     668  0x80000001 0x69b1
10.0.6.2     10.0.6.2     662  0x80000001 0x4bcc
--More--
```

Figure 36: OSPF Database for n4



```
n2# show ip ospf route
===== OSPF network routing table =====
N   10.0.0.0/24      [20] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.2.2, eth1
N   10.0.1.0/24      [10] area: 0.0.0.0
    directly attached to eth0
N   10.0.2.0/24      [10] area: 0.0.0.0
    directly attached to eth1
N   10.0.3.0/24      [40] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.9.1, eth2
N   10.0.4.0/24      [50] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.9.1, eth2
N   10.0.5.0/24      [40] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.9.1, eth2
N   10.0.6.0/24      [20] area: 0.0.0.0
    via 10.0.1.1, eth0
N   10.0.7.0/24      [30] area: 0.0.0.0
    via 10.0.1.1, eth0
    via 10.0.9.1, eth2
N   10.0.8.0/24      [20] area: 0.0.0.0
    via 10.0.9.1, eth2
--More--
```

Figure 37: OSPF Route for n4

Comments:

- The changes upon disconnecting eth0, eth1, and eth2 from n4 can be seen as in the OSPF database, link 10.0.3.1 has 1 link count instead of 3. N4 is now fully disconnected from the network so all its database is removed. The network computes the new paths to every node.
- The cost of some links changed due to the new route.